

Experiment No. 11

Title:

Perform SQLInjection on any website (HTMLget).

Description:

SQL Injection (SQLi) is a web application vulnerability in which specially crafted input is interpreted as part of an SQL statement instead of being treated only as normal user data. It generally occurs when an application builds database queries from user input without using proper safeguards such as parameterized queries.

A common place where this vulnerability can become visible is a **login form**. Normally, the application should authenticate a user only when the supplied username and password satisfy the intended database query. If the input is not handled securely, however, an attacker may manipulate the logical condition of that query.

For this practical, an **intentionally vulnerable web application, Testfire.net**, was used as the testing environment. The banking login page was selected to demonstrate the SQL Injection vulnerability. First, normal invalid credentials were entered to observe the application's expected response. The login was unsuccessful, confirming that the application normally rejects incorrect credentials.

The test was then repeated by placing a **tautology-style SQL Injection input** in the username field while providing an incorrect password. The input used was: ' **OR 1=1 --**

Here, OR 1=1 represents a condition that is always true, while -- is used in SQL syntax to indicate that the remaining part of the query should be treated as a comment, depending on the database and query structure.

In the observed test, the application reported **successful login despite the incorrect password**. This indicates that the supplied username input was able to influence the authentication query and demonstrates an **SQL Injection authentication-bypass condition**.

The practical therefore demonstrates how inadequate handling of user input can affect the security of a database-driven login application.

Methodology:

1. Open **Testfire.net** and navigate to the banking login page.
2. First, enter a **random/invalid username and password** and submit the form.
3. Observe that the login is unsuccessful. This establishes the normal behaviour of the login system.
4. In the **Username** field, enter the SQL Injection test input: **' OR 1=1 --**
5. In the **Password** field, enter a random/incorrect password.
6. Submit the login form and observe the application's response.
7. The application was observed to display a **successful login** even though an incorrect password was supplied.
8. Record the result and capture screenshots showing the test input and the resulting response.
9. Conclude that the login functionality demonstrates an **SQL Injection vulnerability** in the controlled testing environment.

Output:

Observation:

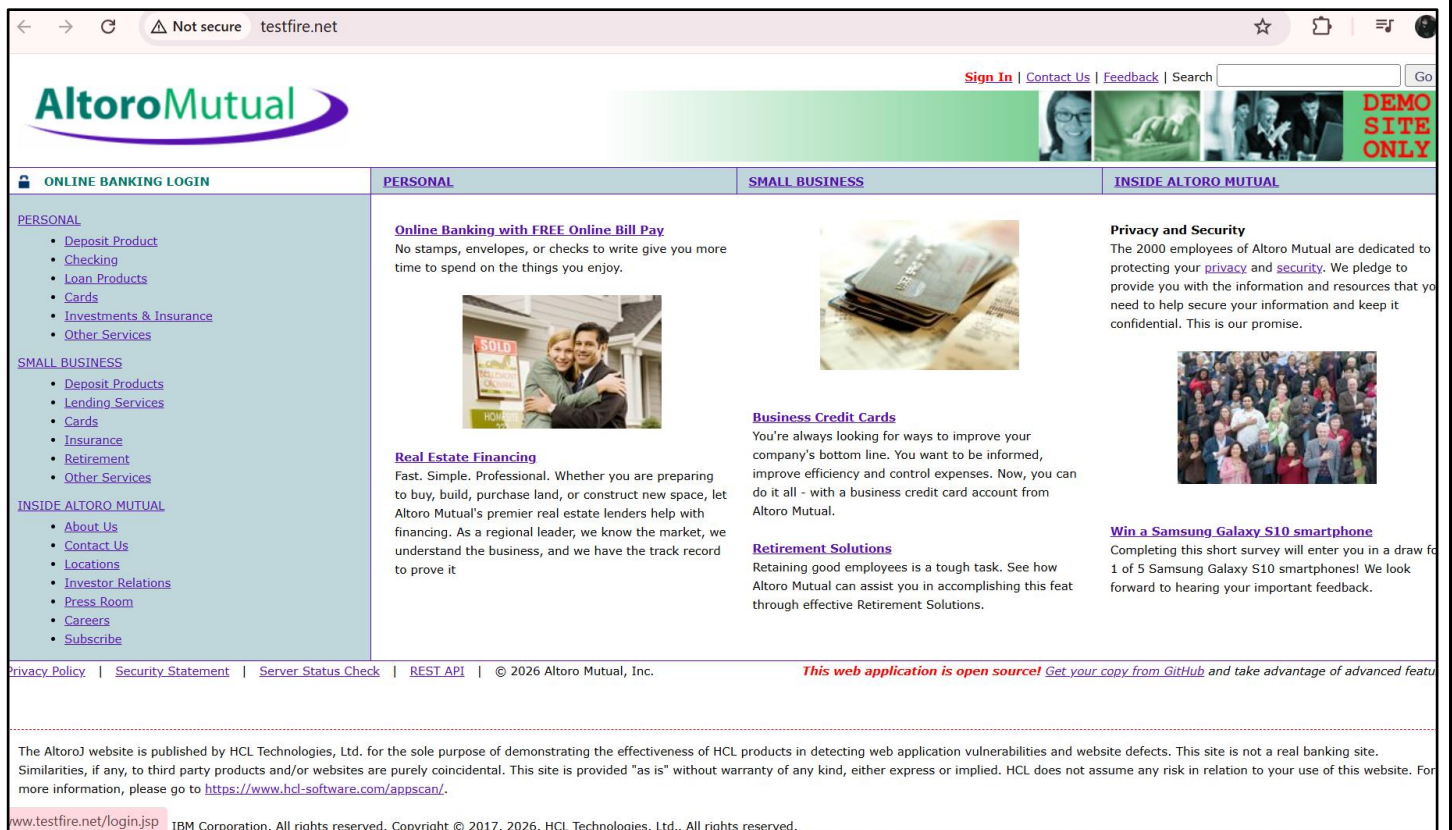
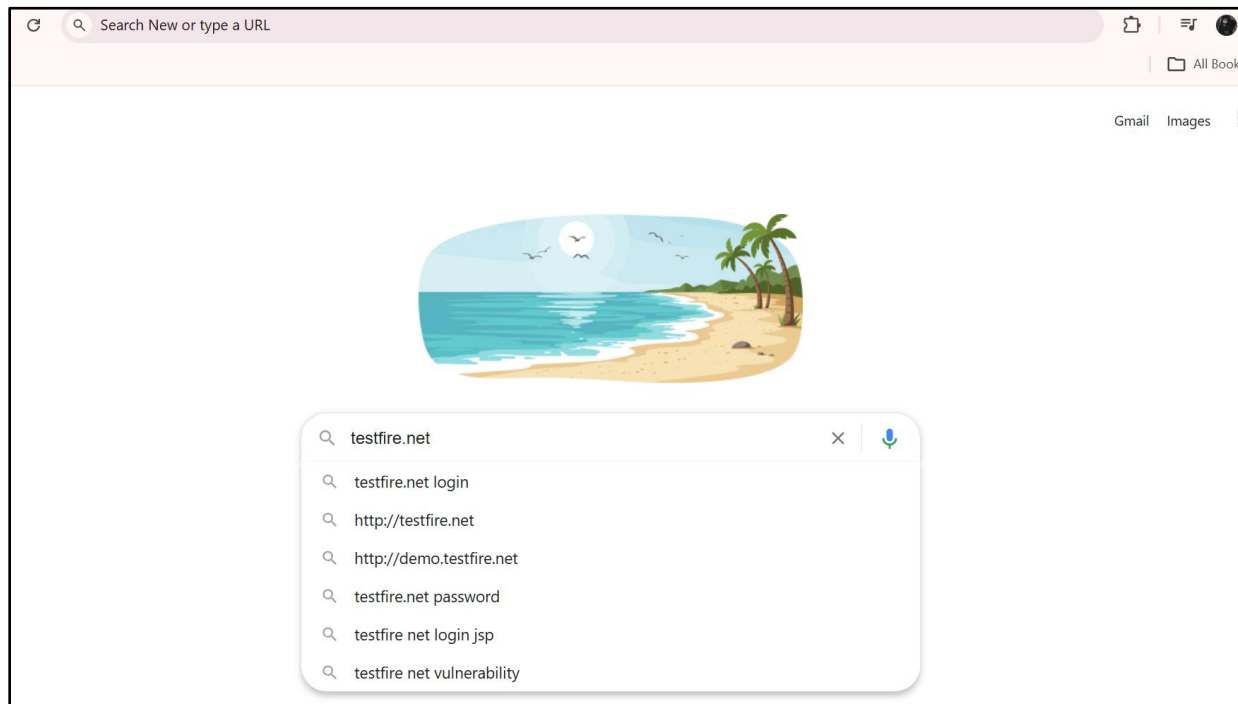
Initially, random and incorrect login credentials were entered into the banking login form. The application did not authenticate the user, which established the expected behaviour when invalid credentials were supplied.

After that, a SQL Injection test was performed by modifying the **username input with a tautology-style SQL expression** while keeping the password incorrect/random. The purpose was to check whether the username input was being incorporated into the backend SQL query without sufficient protection.

During the test, the application responded with a **successful login** even though a valid password had not been entered. This is significant because authentication should normally require both valid username and password information.

The observed behaviour demonstrates that the application's login mechanism was vulnerable to **SQL Injection-based authentication bypass**. The result suggests that the supplied input

changed the logical behaviour of the database query and caused the application's authentication condition to evaluate differently than intended.



AltoroMutual

[Sign In](#) | [Contact Us](#) | [Feedback](#) | [Search](#)

DEMO
SITE
ONLY

ONLINE BANKING LOGIN

PERSONAL

SMALL BUSINESS

INSIDE ALTORO MUTUAL

PERSONAL

- Deposit Product
- Checking
- Loan Products
- Cards
- Investments & Insurance
- Other Services

SMALL BUSINESS

- Deposit Products
- Lending Services
- Cards
- Insurance
- Retirement
- Other Services

INSIDE ALTORO MUTUAL

- About Us
- Contact Us
- Locations
- Investor Relations
- Press Room
- Careers
- Subscribe

Online Banking Login

Login Failed: We're sorry, but this username or password was not found in our system. Please try again.

Username: Apeksha Kharik

Password: *****

Login

[Privacy Policy](#) | [Security Statement](#) | [Server Status Check](#) | [REST API](#) | © 2026 Altoro Mutual, Inc.

This web application is open source! [Get your copy from GitHub](#) and take advantage of advanced fe

AltoroMutual

[Sign In](#) | [Contact Us](#) | [Feedback](#) | [Search](#)

DEMO
SITE
ONLY

ONLINE BANKING LOGIN

PERSONAL

SMALL BUSINESS

INSIDE ALTORO MUTUAL

PERSONAL

- Deposit Product
- Checking
- Loan Products
- Cards
- Investments & Insurance
- Other Services

SMALL BUSINESS

- Deposit Products
- Lending Services
- Cards
- Insurance
- Retirement
- Other Services

INSIDE ALTORO MUTUAL

- About Us
- Contact Us
- Locations
- Investor Relations
- Press Room
- Careers
- Subscribe

Online Banking Login

Username: ' OR 1=1 --

Password: *****

Login

[Privacy Policy](#) | [Security Statement](#) | [Server Status Check](#) | [REST API](#) | © 2026 Altoro Mutual, Inc.

This web application is open source! [Get your copy from GitHub](#) and take advantage of advanced fe

Computer Security (CO23114)



[Sign Off](#) | [Contact Us](#) | [Feedback](#) |



 [MY ACCOUNT](#)

[PERSONAL](#)

[SMALL BUSINESS](#)

[INSIDE ALTORO MUTUAL](#)

I WANT TO ...

- [View Account Summary](#)
- [View Recent Transactions](#)
- [Transfer Funds](#)
- [Search News Articles](#)
- [Customize Site Language](#)

ADMINISTRATION

- [Edit Users](#)

Hello Admin User

Welcome to Altoro Mutual Online.

View Account Details:

Congratulations!

You have been pre-approved for an Altoro Gold Visa with a credit limit of \$10000!

Click [Here](#) to apply.

[Privacy Policy](#) | [Security Statement](#) | [Server Status Check](#) | [REST API](#) | © 2026 Altoro Mutual, Inc.

This web application is open source! [Get your copy from GitHub](#) and take advantage

The AltoroJ website is published by HCL Technologies, Ltd. for the sole purpose of demonstrating the effectiveness of HCL products in detecting web application vulnerabilities and website defects. This site is not a real bank and if any, to third party products and/or websites are purely coincidental. This site is provided "as is" without warranty of any kind, either express or implied. HCL does not assume any risk in relation to your use of this website. For more information, please go to <https://www.hcl-software.com/appscan/>.

Copyright © 2008, 2017, IBM Corporation, All rights reserved. Copyright © 2017, 2026, HCL Technologies, Ltd., All rights reserved.

Conclusion:

The SQL Injection vulnerability was successfully demonstrated on the intentionally vulnerable **Testfire.net** application. The successful login using a SQL Injection test input, despite providing an incorrect password, showed that improperly handled input could alter the intended authentication logic.